Guía de comprensión y reconstrucción progresiva de «Puente»

1. Propósito

Esta guía explica cómo estudiar la implementación **ya resuelta** del repositorio y cómo reconstruir su razonamiento mediante práctica autónoma. El punto de partida es el código completo, los datos observados, los artefactos y la evidencia conservados en esta entrega.

El trabajo recomendado tiene dos espacios separados:

1. **Repositorio de referencia.** Se lee para entender decisiones, contratos y resultados conservados. No se usa como zona de experimentos.
2. **Copia de práctica.** Se duplica el proyecto completo y allí se cambian funciones, fixtures, configuración o artefactos. Las pruebas destructivas y las promociones se limitan a esa copia.

El objetivo no es memorizar el código ni obtener de nuevo una cifra concreta. Es poder responder, con evidencia:

- qué estimando se calcula y bajo qué supuestos;
- cómo viajan los datos hasta el ATE, la tarjeta, el monitor y la API;
- qué comprueba cada diagnóstico y qué no puede demostrar;
- qué archivos cambia cada comando;
- cuándo el sistema devuelve un número y cuándo se abstiene.

La entrega contiene los CSV observados de `synthetic/ihdp`, pero **no incluye el generador del dataset IHDP**. Por tanto, reconstruir el flujo significa partir de los CSV incluidos y regenerar estimaciones y artefactos en una copia, no volver a crear el DGP. `cobertura.py` simula otro DGP ilustrativo y tampoco genera IHDP.

Recorrido de práctica

Conviene hacer varias pasadas, con un propósito distinto en cada una:

1. **Mapa:** seguir entradas, salidas y dependencias sin editar.
2. **Contratos:** predecir qué exige cada función y comprobarlo con ejemplos pequeños.
3. **Método:** derivar a mano AIPW, diagnósticos, sensibilidad y reglas de la puerta.
4. **Sistema:** alterar una sola condición en la copia y observar el efecto en registro, monitor y servicio.
5. **Transferencia:** diseñar otro escenario sin reutilizar supuestos de IHDP por inercia.

En todos los comandos de esta guía se distingue entre:

- **Lectura:** inspecciona archivos o hashes; no ejecuta el flujo.
- **Comprobación sin promoción:** ejecuta pruebas o validaciones, pero no crea una versión ni cambia el champion.
- **Escritura sin promoción:** genera o reemplaza artefactos no versionados.
- **Promoción:** crea una versión y/o cambia el champion.
- **Servicio:** arranca un proceso o construye una imagen; no promueve por sí mismo.

Las instrucciones de regeneración son para la **copia de práctica**. Leer la guía no requiere ejecutarlos.

2. Arquitectura

Responsabilidad del conjunto

El prototipo conecta validación de datos, identificación algebraica, estimación, análisis de sensibilidad, publicación versionada, monitorización y consumo HTTP. Su límite deliberado es trabajar con datos sintéticos y artefactos JSON locales; no es un producto clínico o laboral ni un servicio preparado para exposición pública.

```
observed\_{train,validation,test}.csv + splits.json + identification.json
                                |
                                v
                      datos.py / EsquemaCausal
                                |
                                v
                 estimacion.py + sensibilidad.py
                                |
             +------------------+------------------+
             |                  |                  |
             v                  v                  v
 artifacts/estimacion.json  sensibilidad.json     placebo.json
             |
             v
       registro.py  ----\> artifacts/registro/vNNNN/
             |               estimacion.json
             |               tarjeta.json
             |               manifest.json
             +-----------\> alias\_champion.json + registro/champion
                                |
                                v
             monitor.py -\> artifacts/monitor\_estado.json
                                |
                                v
          servicio\_core.py + servicio.py -\> respuesta o abstención
```

No existe un módulo central que orqueste todo el flujo. El `Makefile` encadena `verificar → estimar → registrar → ventanas → humo`. Cada módulo tiene una frontera propia y también puede importarse desde pruebas. La función `estimar()` calcula y devuelve un diccionario; la ejecución de `estimacion.py` como programa escribe los tres artefactos principales.

Contratos entre bloques

Productor	Salida	Consumidor	`datos.py`	columnas numéricas y validaciones	estimación y monitor
`estimacion.py`	`artifacts/estimacion.json`, `sensibilidad.json`, `placebo.json`	registro, pruebas y lectura analítica	`registro.py`	`vNNNN/{estimacion,tarjeta,manifest}.json`, alias y champion	monitor y servicio
`monitor.py`	`artifacts/monitor_estado.json`	servicio	`servicio_core.py`	diccionarios de respuesta o abstención	endpoints FastAPI y humo

Decisiones importantes:

- El servicio **solo consume el bundle JSON** del champion; no lee CSV ni necesita DoWhy o pandas.
- Registro y monitor son etapas separadas. La cadena registro ⟶ monitor **no es transaccional** y no tiene rollback.
- Las salidas externas son JSON, no `pickle` ni `joblib`.
- La integridad se basa en recomputar hashes SHA-256; detecta diferencias, pero no firma ni autentica la procedencia.

Ruta incremental de construcción

En la **copia de práctica**, construye y comprueba una etapa antes de avanzar. Las validaciones de datos, humo y consultas no promueven; estimación y monitor escriben sin promover, y registro sí crea y promueve una versión. No conviertas esta secuencia en una única orden opaca ni pases a promoción sin inspeccionar los observables.

Paso a comprobar	Artefacto u observable	Criterio antes de avanzar	Datos: esquema, hashes y pared	salida de `datos.py` y `splits.json`	esquema correcto, hashes coincidentes, ausencia de `truth_train.csv` y `n = 448` declarado para `train`
Estimación	`artifacts/{estimacion,sensibilidad,placebo}.json`	existen los tres JSON y la estimación contiene `ate_aipw`, `ic_aipw`, `rangos_covariables`, `estratos`, `identifi-`	Registro	`artifacts/registro/vNNNN/` y dos punteros	existen `estimacion.json`, `tarjeta.json` y `manifest.json` `alias_champion.json`

		cacion, sensibilidad y placebo			y registro/champion apuntan a la versión revisada; monitor todavía debe evaluarse
Monitor posterior	artifacts/monitor_estado.json	version coincide con el champion; revisar accion y estado_puerta de la ventana de producción, sin sustituirlos por peor_accion	Humo	salida de humo.py	aparece HUMO: OK no implica una promoción adicional
Servicio	GET /salud antes de GET /efecto	estado es ok, integridad_valida es true y version es la esperada; también revisar estado_puerta antes de interpretar un número			

Un manifest abreviado muestra el tipo de contrato que debe leerse, no una plantilla para inventar campos:

```
{
  "version": "v0008",
  "schema\_version": "1.2",
  "puerta": "pasa",
  "archivos": {
    "estimacion.json": "\<SHA-256 de estimacion.json\>",
    "tarjeta.json": "\<SHA-256 de tarjeta.json\>"
  }
}
```

Los campos reales de `v0008` incluyen también `dataset_sha256`, hashes duplicados por conveniencia y `alias_version` el esquema de respuesta debe comprobarse en el artefacto concreto. La puerta inicial del manifest no sustituye al monitor posterior.

Práctica autónoma

Dibuja el grafo sin mirar el diagrama anterior. Para cada arista, anota formato, archivo y condición de fallo. Después responde: ¿qué ocurre si falta `estimacion.json`?, ¿y si el monitor pertenece a otra versión?, ¿y si el alias apunta a un manifest modificado? Comprueba las respuestas leyendo los módulos, sin cambiar artefactos.

3. Preparación multiplataforma

Requisitos y entorno

La instalación oficial usa `uv` y Python 3.13. Los scripts instalan `uv` si falta, instalan el intérprete, recrean `.venv` con `--clear`, instalan `requirements.txt` y ejecutan `uv pip check`.

La descarga inicial de `uv` ejecuta código remoto: `curl … | sh` en Unix (o `wget … | sh`) e `Invoke-RestMethod … | Invoke-Expression` en PowerShell. Revisa el instalador antes de ejecutarlo o preinstala `uv` desde una fuente confiable. Si prefieres administrar el entorno sin esos instaladores, usa la instalación manual alternativa del README.

Prerrequisitos según la tarea: `make` para atajos; `curl` (o `wget`) para el instalador y `curl`/`curl.exe` para HTTP; `shasum` para `make evidencia` y para la verificación indicada; `sha256sum` solo como alternativa al comprobar manualmente la lista; Graphviz (`dot`) para `make dag` Typst para `make cierre` y Docker con el plugin Compose para la ruta de contenedor. **El objetivo `make evidencia` invoca concretamente `shasum`: tener `sha256sum` no sustituye esa dependencia al ejecutarlo.** Ninguna herramienta opcional debe instalarse solo para leer los JSON.

En macOS o Linux, desde la raíz del proyecto:

```
bash setup.sh
```

En Windows PowerShell:

```
powershell -ExecutionPolicy Bypass -File setup.ps1
```

Efecto: escribe o recrea `.venv` no arranca el servicio, no estima, no registra y no promueve. `--clear` descarta paquetes adicionales y personalizaciones del entorno anterior. La instalación puede descargar software y dependencias.

`requirements.txt` fija las dependencias directas, pero no constituye un lockfile completo de dependencias transitivas. `uv pip check` comprueba coherencia del entorno instalado, no igualdad byte a byte entre instalaciones.

Intérprete y `PATH`

El `Makefile` usa `PYTHON ?= python3` y `UVICORN ?= uvicorn`: ambos se resuelven mediante `PATH`. Hay dos opciones coherentes:

- activar `.venv` antes de usar `make` o `uvicorn` y, por separado, fijar `SIC_ESCENARIO`
- llamar a rutas directas como `.venv/bin/python` o `.\.venv\Scripts\python.exe`.

Activar el entorno **no fija el escenario**. En macOS/Linux usa `export SIC_ESCENARIO=synthetic/ihdp` antes de los atajos `make`, o antepón `SIC_ESCENARIO=synthetic/ihdp` a una orden. En Windows PowerShell usa `$env:SIC_ESCENARIO = «synthetic/ihdp»` y las rutas `.\.venv\Scripts\… make` solo es una opción si esa herramienta está disponible. Ejecuta `make` sin `-j`: los prerrequisitos de un objetivo no tienen garantizado un orden útil bajo paralelismo y esta cadena depende de etapas anteriores.

En macOS/Linux:

```
source .venv/bin/activate
python --version
```

En Windows PowerShell:

```
.\\.venv\\Scripts\\Activate.ps1
python --version
```

No mezcles un intérprete global con paquetes instalados en `.venv`. Si un módulo parece ausente, comprueba primero qué ejecutable se está resolviendo.

Arranque sin regeneración

Los scripts de arranque seleccionan explícitamente `synthetic/ihdp` y usan el champion conservado. **Arrancan el servicio, pero no ejecutan estimación, registro ni promoción.**

```
bash arrancar.sh
```

```
powershell -ExecutionPolicy Bypass -File arrancar.ps1
```

También puede usarse la ruta directa en macOS/Linux:

```
SIC\_ESCENARIO=synthetic/ihdp .venv/bin/uvicorn servicio:app --host 127.0.0.1 --port 8000
```

y en Windows PowerShell:

```
\$env:SIC\_ESCENARIO = "synthetic/ihdp"
.\\.venv\\Scripts\\uvicorn.exe servicio:app --host 127.0.0.1 --port 8000
```

Detén el proceso con Ctrl-C. Reinicia tras cambiar champion o escenario cuando ya exista una carga válida cacheada.

Comprobación

Antes de experimentar, verifica que la copia contiene código, `data_benchmark_causal/`, `artifacts/`, `evidencia/` y `.venv` funcional. La instalación no sustituye a las pruebas de contratos descritas en la sección 13.

4. Pregunta y datos

Pregunta causal

La pregunta implementada es:

> ¿Cuál es el efecto promedio de `treatment` sobre `outcome` en la población representada por `observed_train.csv` de `synthetic/ihdp`, bajo los supuestos causales declarados?

El estimando principal es el **ATE superpoblacional** `E[Y(1)-Y(0)]`, expresado en puntos del test cognitivo: se refiere a la población sintética que las **448 unidades de `train`** representan, no afirma un cambio para cada individuo. No es el ATT, una recomendación de intervención ni un efecto individual. La unidad de análisis es una fila del escenario sintético; el IC acompaña al efecto promedio bajo el estimador y sus supuestos. Si el objetivo fuera el SATE de esas 448 unidades, habría que declararlo expresamente y reinterpretar la inferencia, en vez de tratar esa muestra fija como la superpoblación sin más.

Datos incluidos

`synthetic/ihdp` contiene 747 unidades en total, repartidas en `train`, `validation` y `test`. El ATE principal se estima en `train`, con **n = 448**. Cada partición observada contiene:

- `treatment`: tratamiento binario `D`
- `outcome`: resultado `Y`
- 25 covariables `X`, declaradas pre-tratamiento;
- `row_id`, que se conserva como identificador en el CSV pero no se convierte en covariable.

Las 25 covariables son `bw`, `b.head`, `preterm`, `birth.o`, `nnhealth`, `momage`, `sex`, `twin`, `b.marr`, `mom.lths`, `mom.hs`, `mom.scoll`, `cig`, `first`, `booze`, `drugs`, `work.dur`, `prenatal`, `ark`, `ein`, `har`, `mia`, `pen`, `tex` y `was`.

El paquete es íntegramente sintético y está inspirado en un benchmark estilo IHDP. No reproduce las covariables reales del ensayo ni autoriza conclusiones clínicas. La motivación y las diferencias con otros usos de «IHDP» se detallan en pregunta_ihdp.md.

Responsabilidad

`datos.py` define el contrato de roles y carga únicamente `observed_<particion>.csv`. La pregunta se declara antes de interpretar efectos; `interventions.json` y cualquier verdad reservada no se usan para estimar, seleccionar cortes o calibrar umbrales.

Funciones clave reales

- `EsquemaCausal.columnas()` y `EsquemaCausal.validar()`: roles, columnas ausentes, duplicidad de roles y variación de `D`.
- `leer(particion=«train»)`: lectura de una partición observada.
- `verificar_splits()`: recomputa hashes de `train`, `validation` y `test`.
- `comprobar_pared()`: exige la ausencia literal de `truth_train.csv`.

Entradas y salidas

La entrada está formada por los CSV y `splits.json`. `leer()` devuelve un diccionario de arrays numéricos. `EsquemaCausal` produce la lista ordenada `treatment`, `outcome`, 25 `X` que utilizan los modelos.

Al ejecutar `datos.py`, los recuentos de filas impresos junto a los hashes proceden de **`splits.json`**, no de un nuevo conteo de cada CSV. El hash sí se recalcula sobre los bytes de cada archivo.

Decisión de diseño

El esquema se declara por escenario en `COVARIABLES` y `RESULTADO`, en lugar de inferir causalmente los roles a partir de nombres o correlaciones. Esto evita que una columna desempeñe dos roles y hace explícito qué entra en el ajuste.

La «pared» comprueba una condición concreta y auditable: `truth_train.csv` no existe en el directorio del escenario. **No es una pared criptográfica general** ni impide por sí sola que otra verdad se copie con otro nombre o que el código se modifique para leerla.

Límites

Validar columnas y hashes no prueba ignorabilidad, positividad poblacional, SUTVA ni calidad científica. Un CSV íntegro puede contener una mala especificación o una fuga con otro nombre. La ausencia de `truth_train.csv` tampoco sustituye una revisión de imports y accesos a datos.

Comprobación y práctica

Comprobación sin promoción en macOS/Linux:

```
SIC\_ESCENARIO=synthetic/ihdp .venv/bin/python datos.py
```

Windows PowerShell:

```
\$env:SIC\_ESCENARIO = "synthetic/ihdp"
.\\.venv\\Scripts\\python.exe datos.py
```

En la copia, formula primero el esquema y el estimando. Después crea una partición mínima independiente con tratamiento constante y comprueba que `validar()` la rechaza. Añade una columna duplicada en el esquema y explica qué invariante se rompe. Restaura el ejemplo antes de continuar.

5. Rutas y configuración

Responsabilidad

`rutas.py` centraliza la raíz, el escenario y las carpetas de artefactos. La variable `SIC_ESCENARIO` selecciona el dataset en la importación del módulo.

Funciones y valores clave

- `raiz_repo()` asciende desde el archivo hasta encontrar `data_benchmark_causal/`.
- `ESCENARIO` toma `SIC_ESCENARIO` su valor por defecto es **`synthetic/jobs`**.
- `DATOS` apunta al escenario dentro del benchmark.
- `ARTEFACTOS` y `REGISTRO` se ubican junto al código, en `artifacts/` y `artifacts/registro/`.

Esta entrega selecciona **explícitamente `synthetic/ihdp`** en scripts, comandos de comprobación y contenedores. No hay que inferir que el default de `rutas.py` representa el escenario de la entrega.

El servicio puede funcionar sin `data_benchmark_causal/`: si no encuentra la raíz del benchmark, conserva rutas de artefactos relativas al código y consume el bundle JSON. Esto permite la imagen Docker sin CSV.

Entradas, salidas y decisión de diseño

- **Entrada:** `SIC_ESCENARIO` para el monitor de producción también existe `SIC_VENTANA`, tratado en la sección 11.
- **Salida:** objetos `Path` usados por los demás módulos.
- **Diseño:** una sola selección de escenario evita mezclar silenciosamente datos y artefactos de contextos distintos, pero la coherencia final también se apoya en `escenario` y `dataset_sha256`.

La configuración se lee al importar. Cambiar la variable en un proceso ya cargado no reconstruye los módulos ni limpia la caché del servicio: define el valor antes de arrancar y reinicia cuando cambie.

Límites

La variable de entorno no convierte artefactos de jobs en artefactos de IHDP. Tampoco elige automáticamente un champion compatible: el registro, la estimación y el monitor deben corresponder al escenario seleccionado. Evita reutilizar una estimación parcial de otro contexto.

Comprobación y práctica

Lee `rutas.py` y escribe a mano las rutas esperadas para ambos escenarios. En la copia, cambia `SIC_ESCENARIO` antes de una importación y registra qué archivos serían entradas y cuáles salidas. No promuevas: el objetivo es detectar dependencias de configuración, no producir otra versión.

6. Integridad y separación de la verdad

La integridad aparece en tres niveles que no deben confundirse:

Nivel	Mecanismo	Qué demuestra	Qué no demuestra	Splits	`verificar_splits()` compara SHA-256 de cada CSV con `splits.json`	los bytes coinciden con lo declarado	origen auténtico o validez causal
Entrenamiento	`dataset_sha256` acompaña estimación y tarjeta	qué `observed_train.csv` se usó	que el DGP sea correcto	Bundle	`manifest.json`, alias y `verificar_integridad()`	estimación y tarjeta no difieren de sus hashes	autenticación, firma o seguridad frente a un atacante con escritura

La separación de la verdad añade una regla científica: no leer `truth_*.csv`, `y0`, `y1`, `tau` ni los valores de `interventions.json` para estimar o decidir. La prueba estática de `pruebas_estimacion.py` revisa lecturas reales de `interventions` en `estimacion.py` y `sensibilidad.py` es una salvaguarda acotada, no una demostración formal de ausencia de fugas.

El manifest cubre `estimacion.json` y `tarjeta.json`. **No hashea `monitor_estado.json`**, que es un archivo operativo posterior y versionado mediante su campo `version`.

Práctica autónoma

En una copia temporal del bundle, no en el registro de referencia:

1. recompute el hash de un archivo y compáralo con el manifest;
2. cambia un byte en la copia de `tarjeta.json`
3. predice qué verificación fallará y si el servicio devolverá número;
4. restaura la copia y separa claramente el fallo de integridad del estado de la puerta.

La función `demo_mutacion()` ilustra una mutación creando y luego eliminando `estimacion.json.mutado` dentro de una versión. No es una lectura inocua: si se estudia su ejecución, debe hacerse únicamente en una copia temporal.

7. Identificación

Responsabilidad

La identificación se declara mediante un DAG y un conjunto de ajuste. La estrategia es backdoor sobre covariables observadas pre-tratamiento, no selección automática de predictores. Bajo ignorabilidad condicional, positividad y consistencia/SUTVA, el ATE se escribe:

$$\tau = E[Y(1) - Y(0)] = E_X(E[Y \mid X, D = 1] - E[Y \mid X, D = 0]).$$

Esta fórmula separa el parámetro identificado de su estimación. DoWhy opera sobre el DAG y los metadatos declarados: no prueba que esos supuestos se cumplan. Los diagnósticos pueden revelar problemas, pero no sustituyen la defensa del diseño.

Funciones clave reales

- `_grafo_backdoor_networkx()` añade `X → D`, `X → Y` para cada covariable y `D → Y`.
- `identificacion_dowhy()` construye el modelo, identifica el estimando y compara el conjunto con el benchmark.
- `validacion_particiones()` reestima AIPW en `train`, `validation` y `test` como comprobación de estabilidad, no como identificación.

Un fragmento mínimo de la construcción es:

```
for c in esquema.covariables:
    g.add\_edge(c, esquema.tratamiento)
    g.add\_edge(c, esquema.resultado)
g.add\_edge(esquema.tratamiento, esquema.resultado)
```

Entradas y salidas

Entradas: columnas observadas, `EsquemaCausal` e `identification.json`. **Salida:** un diccionario dentro de `estimacion.identificacion` con aristas, grafo, conjunto de ajuste, indicadores y resúmenes de DoWhy.

DoWhy se invoca con dos métodos complementarios:

- `method_name=«default»`: produce el `IdentifiedEstimand` y el conjunto backdoor;
- `method_name=«id-algorithm»`: produce una expresión algebraica ID complementaria.

`identificacion_pasa` exige un conjunto no vacío y coincidencia con el primer `valid_adjustment_sets` declarado por el benchmark. Esto comprueba **coherencia con `identification.json`**, no ignorabilidad empírica.

Decisión de diseño

El DAG se deriva del esquema, de modo que el ajuste y la estimación usan las mismas covariables. La comparación con el benchmark detecta discrepancias de implementación. La expresión ID se conserva como explicación algebraica, no como una segunda prueba independiente.

Límites

El grafo no descubre confusores ausentes. DoWhy no puede demostrar `{Y(0),Y(1)} ⊥ D | X`, consistencia/SUTVA ni que todas las covariables sean realmente pre-tratamiento. Un `pasa` significa coherencia algebraica con las declaraciones disponibles.

La validación por particiones tampoco es un *holdout* predictivo: **reestima AIPW en cada partición** y compara ATE e intervalos. El estimando primario sigue siendo el de `train` estabilidad o inestabilidad entre particiones no prueba ni refuta por sí sola ignorabilidad.

Comprobación y práctica

En la evidencia y en `estimacion.json`, localiza `conjunto_ajuste_estimado`, `coincide_valid_adjustment_sets` y `metodos_dowhy`. Después dibuja un confusor no observado `U` con `U → D` y `U → Y`. Explica por qué añadirlo mentalmente al DAG cambia la conclusión aunque el archivo de datos permanezca igual.

Como ejercicio de contraste, formula qué diseño necesitaría un instrumento, un panel o una regresión discontinua y señala por qué ninguno está implementado aquí.

8. Estimación

Responsabilidad

`estimacion.py` calcula la diferencia cruda, un OLS ajustado como comparador, el ATE principal mediante AIPW con cross-fitting, diagnósticos de positividad y balance, dos descripciones de heterogeneidad y una trampa de fuga. También integra identificación, sensibilidad, placebos y particiones en el diccionario final.

Antes de leer la estimación AIPW conviene separar cinco capas; ninguna herramienta de una capa resuelve las demás:

Capa	Pregunta	Herramienta en el proyecto	Límite principal	Estimando	¿Qué contraste, para qué población y en qué unidades?	ATE superpoblacional declarado para la población sintética representada por `train`	no es efecto individual ni recomendación
Identificación	¿Qué supuestos conectan resultados potenciales con datos observados?	DAG, backdoor y DoWhy	coherencia algebraica, no prueba del DAG ni de ignorabilidad	Estimación	¿Cómo se aproxima el funcional identificado?	AIPW con nuisances OOF y comparadores OLS/crudo	depende de regularidad, soporte y estimación adecuada de nuisances
Inferencia	¿Qué incertidumbre muestral se describe?	SE del score e IC normal asintótico aproximado	no ofrece cobertura exacta ni incorpora todo el sesgo	Evaluación	¿Qué tan soportado, estable y útil es el resultado?	diagnósticos, particiones, sensibilidad, placebos y monitor	cada señal es parcial; no forman un certificado agregado

Funciones clave reales

- `ate_ols()`: OLS con covariables, SE, IC, `t` y grados de libertad.
- `aipw_crossfit()`: nuisances OOF y score AIPW.
- `diagnosticos_positividad()`, `ess_kish()` y `smd_ponderado()`: soporte y balance.
- `estratos_heterogeneidad()`: medias de scores por un corte declarado.
- `modelo_individual()`: proyección lineal del score sobre `X`.
- `trampa_score_post()`: columna derivada de `Y` para ilustrar fuga.
- `validacion_particiones()` e `identificacion_dowhy()`: estabilidad y coherencia algebraica.
- `estimar()`: ensambla el resultado; no es un orquestador de registro, monitor o servicio.

Entradas y salidas

Entrada principal: `observed_train.csv` y el esquema. También se leen las otras particiones e `identification.json`.

Salida al importar: `estimar()` devuelve un diccionario con ATE, incertidumbre, comparadores, sensibilidad, placebos, identificación, rangos y modelos.

Salida al ejecutar el archivo: escribe, sin promover:

- `artifacts/estimacion.json`
- `artifacts/sensibilidad.json`
- `artifacts/placebo.json`.

AIPW con cross-fitting

La implementación usa **K = 5** y semilla **4242**. En cada pliegue ajusta fuera de él:

- OLS por brazo para $\mu_1(X)$ y $\mu_0(X)$
- regresión logística para $e(X)$, con covariables estandarizadas.

Predice dentro del pliegue, de modo que cada observación recibe nuisances **out-of-fold**. Después aplica **acotación (llamada trimming en artefactos)** a la propensión, `np.clip(e, 0.05, 0.95)`, y calcula:

$$ \psi_i = \hat\mu_1(X_i)-\hat\mu_0(X_i)

- \frac{D_i{Y_i-\hat\mu_1(X_i)}}{\tilde e_i}

- \frac{(1-D_i){Y_i-\hat\mu_0(X_i)}}{1-\tilde e_i}, \qquad \tilde e_i=\operatorname{clip}(\hat e_i,0.05,0.95). $$

El estimador y su incertidumbre son:

$$\widehat{\mathrm{ATE}} = \frac{1}{n}\sum_{i=1}^{n}\psi_i, \quad \widehat{\mathrm{SE}} = \frac{s_\psi}{\sqrt{n}}, \quad \mathrm{IC}_{95} = \widehat{\mathrm{ATE}} \pm 1.96\widehat{\mathrm{SE}},$$

donde s_ψ es la desviación estándar muestral de los scores (`ddof=1`). El fragmento central se puede contrastar directamente con la fórmula:

```
score = (mu1 - mu0 + D \* (Y - mu1) / e
         - (1 - D) \* (Y - mu0) / (1 - e))
ate = float(score.mean())
se = float(score.std(ddof=1) / np.sqrt(n))
```

`np.clip` **acota o winsoriza propensiones; no elimina filas**. `recortado_n` cuenta propensiones modificadas, no unidades excluidas, y las 448 filas permanecen en el score. El AIPW resultante es regularizado/acotado: puede diferir y sesgarse respecto del score no acotado, y no selecciona automáticamente un ATO.

Cross-fitting evita que un nuisance se ajuste con la misma fila que predice. No elimina confusión ni garantiza buena especificación. La doble robustez exige los supuestos causales, condiciones de regularidad y estimación adecuada de nuisances; al menos una pierna debe estar correctamente especificada en el sentido del resultado de doble robustez. Si ambas fallan, AIPW puede sesgarse.

El IC es una aproximación normal asintótica, no una garantía de cobertura exacta. OOF no es validación predictiva del efecto y cross-fitting no identifica por sí solo, no crea positividad o cobertura y no elimina la dependencia entre pliegues. *Repeated cross-fitting* podría estudiar variabilidad adicional, pero no está implementado. Las particiones de validación reentrenan AIPW en cada partición; no consultan un champion congelado como si hicieran predicción fuera de muestra.

Diagnósticos de soporte y balance

`diagnosticos_positividad()` recibe las propensiones OOF **antes de acotar** y conserva rango, cuantiles, fracción dentro de `[0.05,0.95]` y recuentos de propensiones modificadas. Para estabilizar los pesos, el ESS de Kish y el SMD ponderado usan la propensión acotada.

Con `w_i=D_i/\tilde e_i+(1-D_i)/(1-\tilde e_i)` en la fila correspondiente, el ESS es:

$$\mathrm{ESS} = \frac{\left(\sum_i w_i\right)^2}{\sum_i w_i^2}.$$

El SMD ponderado compara medias ponderadas una covariable a la vez y las normaliza con las varianzas ponderadas de ambos brazos. Es un diagnóstico muestral, no una corrección del ATE ni una prueba de balance poblacional. No certifica balance conjunto ni igualdad de formas o relaciones no lineales.

Lee **ESS, solape, extremos y SMD juntos**. Un ESS alto no garantiza soporte: una asignación casi determinista puede producir pesos poco variables y ESS alto mientras falta comparabilidad. El monitor usa mediciones dentro de sus ventanas; esas señales *in-sample* no son intercambiables con las propensiones OOF del champion.

Estratos y proyección por perfil

`estratos_heterogeneidad()` divide IHDP por `bw` usando la **mediana de `train`**. Cada hoja contiene la media de los scores AIPW, su SE y su IC. `/consulta` devuelve así una **estimación agregada y gruesa de un efecto condicional para una de dos subpoblaciones**, no un efecto individual. Su lectura exige los mismos supuestos causales y positividad dentro del subgrupo.

Dos estimaciones distintas no demuestran heterogeneidad: para sostener un contraste alto–bajo hay que estimar directamente esa diferencia y su incertidumbre. El solapamiento visual de IC tampoco es una prueba de diferencia entre estratos.

`modelo_individual()` ajusta una proyección lineal:

$$\psi_i = \beta_0 + Z_i^\top \beta + \varepsilon_i,$$

con `X` estandarizada. Guarda intercepto, coeficientes, medias, desviaciones, covarianza de coeficientes, `R²`, `n` y semilla. Para un perfil, la API usa ese modelo y la varianza `v'Cov(β)v` para un IC **condicional al ajuste**. Es una proyección de pseudo-scores, no un CATE individual identificado, y el IC no incorpora toda la incertidumbre de los scores ni el error de especificación. El `R²` mide ajuste *in-sample* a esos pseudo-scores, no el porcentaje de heterogeneidad causal explicada.

`trampa_score_post()` crea una variable con `Y`, `D` y ruido. Su atenuación ilustra fuga post-resultado; **no es un placebo**.

Evidencia existente, no resultado garantizado

La versión conservada **`v0008`** contiene, como ejemplo de evidencia ya existente, un ATE AIPW de **5,2 puntos** con IC 95 % **[3,9; 6,4]**. No es una cifra garantizada para otra copia, entorno, modificación o versión. El estudio debe contrastar el artefacto efectivamente cargado y no copiar ese resultado como expectativa universal.

Decisión de diseño

Se prefieren modelos simples y auditables —OLS por brazo y logística— frente a una colección opaca de learners. El mismo score alimenta el ATE y las descripciones de heterogeneidad, manteniendo una cadena común. Los artefactos almacenan el modelo de proyección, no los scores individuales.

Límites

- Propensiones estimadas, acotación y rangos observados no prueban positividad poblacional.
- Los IC usan una aproximación basada en la variabilidad del score; no resuelven mala especificación.
- Coincidencia entre diferencia ingenua, OLS y AIPW no prueba identificación.
- Los rangos marginales no garantizan soporte multivariable.

- La proyección lineal describe heterogeneidad estimada con discriminación posiblemente limitada.

Comprobación y práctica

Antes de ejecutar nada, construye a mano un ejemplo de dos o cuatro filas con `e=0,5`, asigna μ_0, μ_1, `D` e `Y`, y calcula ψ, ATE y SE. Usa luego una función independiente en la copia para contrastar el cálculo con `aipw_crossfit()` en un fixture controlado.

Ejercicios adicionales:

1. Cambia solo la semilla de folds y describe qué debería permanecer invariante y qué puede variar.
2. Introduce deliberadamente una propensión extrema en un fixture y separa el efecto sobre diagnósticos, ESS y score.
3. Demuestra con un ejemplo que la media de scores de un estrato no equivale a un efecto individual.
4. Explica por qué `score_post` es fuga y no control negativo.

Escritura sin promoción, solo en la copia: `SIC_ESCENARIO=synthetic/ihdp .venv/bin/python estimacion.py`. Reemplaza los tres artefactos no versionados indicados arriba; no crea `vNNNN` ni cambia el champion.

9. Sensibilidad y placebos

Responsabilidad

`sensibilidad.py` ofrece tres perspectivas de sensibilidad y dos controles de calibración. Recibe columnas y resultados ya calculados; no lee CSV ni `interventions.json`.

Funciones clave reales

- `e_value_continuo()`
- `valor_robustez()`
- `cotas_manski()`
- `resumen_sensibilidad()`
- `placebo_permutacion()`
- `control_negativo()`.

Entradas y salidas

El resultado se incorpora a `estimacion.json` bajo `sensibilidad` y `placebo`. Al ejecutar `estimacion.py`, también se escriben `artifacts/sensibilidad.json` y `artifacts/placebo.json`. Estas salidas no crean una versión.

E-value continuo aproximado

Para un resultado continuo se estandariza `d=|ATE|/sd(Y)` y se aproxima una razón de riesgos mediante `RR ≈ exp(0.91·d)`. Después se calcula:

$$E = \mathrm{RR} + \sqrt{\mathrm{RR}(\mathrm{RR}-1)}, \quad \mathrm{RR} > 1.$$

Para el intervalo se usa el límite más cercano a cero; si el IC contiene cero, el E-value del límite es 1.

Límite decisivo: la RR es una aproximación para el efecto estandarizado, **no una RR literal**. El E-value no demuestra ausencia de confusión ni cubre todos los errores de especificación, selección, medición, positividad o dependencia.

Valor de robustez de Cinelli–Hazlett

`valor_robustez()` usa `t` y grados de libertad del **OLS ajustado** para obtener la fuerza parcial mínima de confusión que reduciría su coeficiente a cero bajo el modelo lineal de sesgo por omisión.

Es exacto bajo ese marco lineal para OLS. **No es una propiedad exacta del AIPW**, aunque ambos resultados viajen juntos en el bundle.

Cotas de Manski

`cotas_manski()` no presupone ignorabilidad. Descompone las medias potenciales en partes observadas y no observadas y acota estas últimas con un soporte `[L,U]`. Produce identificación **parcial**, no un ATE puntual; el ancho del intervalo del ATE es `U−L`.

Si no se declara soporte externo, usa el mínimo y máximo observados en `train`. Esto es una ilustración: el rango muestral no garantiza el soporte poblacional y puede generar cotas demasiado estrechas.

Placebos

`placebo_permutacion()` permuta `D` **incondicionalmente**, conserva `X` e `Y` y rompe su dependencia con `X` mantiene los pliegues fijos y reestima AIPW. Compara el efecto observado con la distribución de placebos y calcula un valor `p` Monte Carlo aproximado. No es un test general de aleatorización válido para cualquier diseño. `placebo_ok` exige conjuntamente `p <= 0.05` y que el efecto observado quede fuera del percentil 2,5–97,5 de las permutaciones: significa extremo frente a esa distribución artificial, no evidencia causal.

`control_negativo()` permuta una covariable pre-tratamiento —en IHDP, `momage`— y la usa como pseudo-resultado en una clave separada, sin reemplazar la covariable original en `X`. El efecto nulo está construido por esa permutación. `control_negativo_ok` pasa si el IC contiene cero **o** si `|efecto| <= 0,1·sd` de la variable original; el 0,05 y el 0,1 son convenciones del procedimiento, no umbrales causales universales.

Ambos evalúan calibración e implementación. **Ninguno prueba ignorabilidad ni ausencia de confusión**: un procedimiento confundido puede superar un placebo.

Decisión de diseño y comprobación

Las medidas responden preguntas distintas y no se combinan en una supuesta prueba única. La tarjeta y la API conservan sus límites textuales para impedir que un número de sensibilidad se convierta en certificado causal.

Cómo leer conjuntamente una estimación de efectos

La evaluación genuina es una **rúbrica de dimensiones**, no una suma de semáforos ni un *score* agregado. Un fallo o acierto aislado no decide la validez científica.

Dimensión	Qué revisar	Lectura prudente	Estimando	población, contraste, unidad y horizonte	distinguir ATE superpoblacional, SATE, estratos y proyección

Magnitud y relevancia	tamaño en unidades y umbral práctico declarado	este proyecto no preespecifica un umbral de relevancia práctica; el IC solo no demuestra importancia sustantiva	Precisión	SE, ancho y valores compatibles del IC	un IC estrecho puede coexistir con sesgo: precisión no es validez
Identificación	DAG, ignorabilidad, positividad y consistencia/SUTVA	supuestos defendidos, no verificados automáticamente	Soporte y balance	extremos, solape, ESS y SMD conjuntamente	ninguna señal aislada certifica comparabilidad multivariable
Estabilidad y especificación	estimadores, particiones y sensibilidad de decisiones	diferencias orientan revisión; no prueban sesgo eliminado ni diferencias causales	Sensibilidad	E-value, RV, Manski y controles	perspectivas parciales bajo supuestos distintos, no prueba de ausencia de confusión
Calibración	comportamiento de placebos y cobertura Monte Carlo	cada calibración pertenece a su construcción o DGP, no se transfiere a IHDP			

Un IC completamente positivo indica compatibilidad con efectos positivos **bajo el modelo y sus supuestos**, no prueba causalidad. Un IC que incluye cero no prueba efecto nulo; uno ancho puede admitir efectos beneficiosos, nulos o dañinos y ser poco concluyente. Evita sustituir esta lectura por un umbral de `p`, por dicotomías significativo/no significativo o por comparar visualmente solapes de intervalos.

Ejemplo con `v0008`, no resultado garantizado para otra versión:

- El ATE AIPW es **5,157 puntos**, SE **0,626** e IC 95 % **[3,930; 6,384]**. Describe un promedio, no un cambio de 5,157 en cada individuo.
- El contraste crudo es **10,327** y OLS ajustado **4,799**: el ajuste importa descriptivamente, pero esas diferencias no miden cuánto sesgo se eliminó ni validan la especificación.
- El rango OOF es **[0,043; 0,916]**; hubo **1 propensión acotada**, no una fila eliminada. `ESS/n = 0,742` señala pérdida de eficiencia por ponderación. El SMD ponderado máximo baja de **0,429** sin ponderar a **0,113**, una mejora descriptiva que no certifica soporte ni balance conjunto.
- Reentrenando por particiones, `validation` da **1,933 [−2,448; 6,314]** y `test` **1,630 [−3,915; 7,176]**. Son puntos menores e intervalos anchos: estabilidad limitada e imprecisa, no una diferencia demostrada sin estimar el contraste correspondiente.
- Los estratos de `bw` alto y bajo dan **7,155** y **3,159**: sugieren heterogeneidad agregada, pero falta estimar su contraste con incertidumbre. La proyección tiene `R² = 0,095`, bajo, y ese ajuste a pseudo-scores no mide heterogeneidad causal.
- El E-value aproximado es **2,073** para el punto y **1,852** para el límite; el RV **37,4 %** pertenece a OLS, no a AIPW. Manski da **[−37,656; 48,477]**, contiene cero y expresa identificación parcial: no es un IC y, sin ignorabilidad, bajo ese soporte no identifica el signo.

En conjunto, `v0008` es compatible con un efecto ajustado positivo en el DGP sintético bajo los supuestos declarados, con precisión razonable en `train` y sin un fallo inmediato en los controles conservados. La estabilidad entre particiones es limitada e imprecisa; identificación depende de supuestos y los placebos no demuestran ausencia de confusión. Esto no produce una recomendación ni una conclusión sobre niños reales. Que la puerta pase solo indica cumplimiento de umbrales en la ventana operativa seleccionada: no mide calidad científica ni anula los resultados de estrés.

Práctica autónoma:

1. calcula el E-value para un IC que cruza cero y justifica el resultado del límite;
2. reproduce el ejemplo cerrado de Manski usado en `pruebas_estimacion.py` y verifica que el ancho es `U−L`
3. explica por qué el valor de robustez no debe etiquetarse como «RV AIPW exacto»;
4. diseña una situación con confusión real donde un placebo pueda pasar.

Contrasta tus respuestas con las pruebas de fórmulas, sin esperar que esas pruebas validen los supuestos causales.

10. Registro y promoción

Responsabilidad

`registro.py` convierte una estimación en un bundle versionado, verifica su integridad y mantiene el puntero champion. Separa publicación de estimación, pero su CLI promueve antes de que exista una nueva evaluación del monitor.

Funciones clave reales

- `construir_tarjeta()` y `validar_tarjeta()`
- `siguiente_version()` y `registrar()`
- `sha256_archivo()` y `verificar_integridad()`
- `verificar_alias()`
- `promover()` y `champion()`.

Entradas y salidas

Entrada: una estimación compatible con el escenario, con sensibilidad, placebos, diagnósticos y hash del dataset. La CLI reutiliza `artifacts/estimacion.json` si es apta; si falta o no corresponde, puede recalcularla.

Salida: `artifacts/registro/vNNNN/` con:

```
estimacion.json
tarjeta.json
manifest.json
```

Además actualiza `artifacts/alias_champion.json` y `artifacts/registro/champion`. Las versiones son **inmutables por convención**, no por un mecanismo que impida físicamente editarlas.

Tarjeta y manifest

`CAMPOS_TARJETA` exige 15 campos y `version_esquema` es **1.2**:

1. `estimando`
2. `poblacion`
3. `supuestos`
4. `umbrales`
5. `semilla`
6. `dataset_sha256`
7. `estimador`
8. `uso_previsto`
9. `limitaciones`
10. `soporte`
11. `puerta`
12. `sensibilidad`
13. `placebo`
14. `identificacion`
15. `version_esquema`.

El manifest registra hashes de **estimación y tarjeta**, además de versión, esquema, dataset y puerta. No incluye el hash del monitor.

El alias almacena la versión y el hash del manifest. `verificar_alias()` exige que ese hash coincida y que los hashes internos del bundle verifiquen. Así detecta tanto un alias desactualizado como cambios en archivos cubiertos.

Promoción y atomicidad real

`promover()` bloquea exactamente cuando `manifest[«puerta»] == «falla»`. Si es `alerta`, exige `aceptar_alerta=True`.

Solo el puntero de texto `registro/champion` se actualiza mediante escritura temporal y **rename atómico**. `alias_champion.json` se escribe antes y esa escritura no es transaccional. Por tanto, no debe afirmarse que alias y champion cambian como una única operación atómica.

La CLI registra la versión con estado inicial **`pasa`** y la promueve. El monitor se ejecuta después en la cadena del `Makefile`. Registro → monitor no es transaccional y no hay rollback: si la etapa posterior falla o cambia la puerta, la versión ya creada y promovida no se deshace automáticamente.

Decisión de diseño

El bundle contiene todo lo que necesita el servicio y evita serializar modelos binarios. El manifiesto pequeño facilita auditoría. La puerta del manifest y el estado operativo del monitor son fuentes distintas: el servicio prefiere un monitor de la misma versión y cae al manifest si falta o no coincide.

Límites

- La inmutabilidad depende de disciplina de edición.
- Los hashes no autentican al autor ni protegen contra modificaciones coordinadas de archivos y hashes.

- Un estado inicial `pasa` no equivale a una evaluación posterior del monitor.
- Alias y champion pueden quedar inconsistentes ante una interrupción entre escrituras.

Comprobación y práctica

Lectura: inspecciona una versión conservada y relaciona los dos hashes del manifest con los archivos. Después verifica que el hash del manifest coincida con el alias.

Promoción, solo en la copia: `SIC_ESCENARIO=synthetic/ihdp .venv/bin/python registro.py`. Crea una nueva `vNNNN`, puede recalcular la estimación, ejecuta sus demostraciones internas y cambia alias/champion. No es una comprobación de solo lectura.

En la copia, diseña un experimento separado para cada condición: puerta `falla`, `alerta` sin aceptación, manifest modificado y alias modificado. Predice el resultado antes de llamar a `promover()` o `verificar_alias()`. No edites la entrega de referencia para crear estos casos.

11. Monitor y puerta causal

Responsabilidad

`monitor.py` genera ventanas de estrés, mide señales, clasifica acciones y escribe el estado operativo del champion. No recalibra umbrales ni vuelve a ajustar AIPW durante la sensibilidad de umbrales.

Funciones clave reales

- `psi()`
- `ventana_sana()`, `ventana_deriva()`, `ventana_positividad()`, `ventana_balance()` y `ventana_confusor_oculto()`
- `medir_ventana()` y `evaluar_ventana()`
- `clasificar_ventana()`
- `umbrales_escalados()` y `sensibilidad_umbrales()`.

Entradas y salidas

Entradas: `train`, el champion resuelto por `champion()`, su tarjeta y su estimación. **Salida:** `artifacts/monitor_estado.json`, con versión, ventana de producción, acción, puerta, resultados de las cinco ventanas y sensibilidad de umbrales. No crea otra versión ni promueve.

Las ventanas son:

Ventana	Transformación o propósito	`sana`	submuestra sin reemplazo de la misma población
`deriva`	desplaza una covariable configurada	`positividad`	selecciona una asignación casi determinista para tensionar solape
`balance`	separa tratados y controles en una covariable	`confusor_oculto`	introduce una `U` no observada que afecta a `D` e `Y`

Reglas de clasificación

La prioridad de acciones es:

```
abstenerse \> revisar\_identificacion \> reentrenar \> alertar \> continuar
```

`clasificar_ventana()` aplica estas reglas en orden:

Señal	Acción	Puerta	`D` no varía	`abstenerse`	`falla`
solape por debajo del mínimo	`abstenerse`	`falla`	placebo o control negativo falla	`revisar_identificacion`	`revisar_identificacion`
PSI máximo supera el umbral	`reentrenar`	`reentrenar`	**SMD ponderado** supera el umbral	`alertar`	`alerta`
inestabilidad del ATE supera el umbral	`alertar`	`alerta`	resto	`continuar`	`pasa`

El SMD sin ponderar se informa, pero no clasifica. El umbral de estabilidad combina `2·SE` del champion con una fracción del valor absoluto de su ATE.

Producción y estrés

Solo `SIC_VENTANA` determina qué ventana fija la puerta de producción. El default es **`sana`**; un nombre desconocido vuelve a `sana`.

Las otras ventanas describen estrés. En particular, una ventana de `positividad` adversa **no bloquea el servicio por sí sola** si no es la ventana de producción. `peor_accion` resume la acción más severa observada, pero no sustituye a `accion` y `estado_puerta` de producción.

Los umbrales de la tarjeta son **convenciones iniciales**. `sensibilidad_umbrales()` escala los cortes con factores `0.8`, `1` y `1.2` — exploración de **±20 %**— y reclasifica medidas ya calculadas. **No reajusta AIPW**, no optimiza los umbrales y no demuestra calibración universal.

Decisión de diseño

`medir_ventana()` separa medición de clasificación. Así, la sensibilidad de umbrales reutiliza exactamente las mismas señales y evita atribuir a un cambio de umbral una variación debida a un nuevo ajuste.

Límites

Las ventanas son escenarios sintéticos de estrés, no una muestra de todos los fallos futuros. La ventana `confusor_oculto` no convierte lo no observado en detectable de forma general. Un PSI bajo o un ATE estable tampoco descarta confusión.

Comprobación y práctica

Escritura sin promoción, solo en la copia: `SIC_ESCENARIO=synthetic/ihdp .venv/bin/python monitor.py`. Requiere un champion disponible y reemplaza `monitor_estado.json`.

Para practicar sin depender de artefactos nuevos, crea un diccionario de medidas artificiales y llama a `clasificar_ventana()` con casos frontera. Cambia una sola señal por ejercicio. Después usa `sensibilidad_umbrales()` y verifica que las medidas no cambian, solo la clasificación.

12. Servicio y abstención

Responsabilidad

`servicio_core.py` carga y verifica el champion, aplica la puerta y construye respuestas. `servicio.py` es un envoltorio FastAPI para JSON, texto y formulario. La abstención es una respuesta causal deliberada: HTTP 200 sin efecto o proyección numérica.

Funciones clave reales

- `_cargar()` y `_estado_puerta()`
- `_abstencion_por_puerta()`
- `salud()`, `efecto()` y `consulta()`
- `heterogeneidad()` y `efecto_individual()`
- `campos_formulario()` y `formatear()`.

Carga, caché y puerta

`_cargar()` resuelve `champion()`, verifica alias e integridad y carga `manifest.json`, `estimacion.json` y `tarjeta.json`. No lee CSV.

Una carga válida queda cacheada indefinidamente en el proceso. **Reinicia después de cambiar champion o escenario** y también cuando quieras recargar un bundle válido distinto. Una carga fallida no se cachea: devuelve un estado que fuerza abstención y permite reintentar después de reparar el artefacto.

El fallback exacto distingue dos casos. Si `monitor_estado.json` **falta o pertenece a otra versión**, el servicio usa `manifest.puerta`. Si el JSON del monitor **está corrupto o no es un objeto JSON**, la excepción invalida toda la carga: `integridad_valida` es `false`, `_estado_puerta()` devuelve `falla` y el servicio se abstiene. Si el monitor es un objeto válido para la versión, su `estado_puerta` tiene prioridad.

Bloquean los números:

- integridad rota o carga inválida;
- puerta activa exactamente `falla`
- perfil fuera de soporte o incompleto según el contrato del endpoint.

La puerta `alerta`, `reentrenar` o `revisar_identificacion` **no bloquea por sí sola** la respuesta numérica. Debe conservarse en la interpretación operativa; no equivale a `pasa`.

Contratos de los endpoints

Endpoint	Entrada	Respuesta dentro de soporte	`GET /efecto`	ninguna	ATE AIPW, IC, versión, identificación, sensibilidad y placebos
`POST /consulta`	para IHDP exige `bw` otras covariables presentes se contrastan con sus rangos	media de scores AIPW del estrato de `bw` e IC	`POST /heterogeneidad`	exige las 25 covariables	`proyeccion`, IC condicional, R^2 y advertencias
`POST /efecto_individual`	mismo perfil	alias deprecado de `/heterogeneidad`	`GET /salud`	ninguna	estado, versión, puerta, integridad y resúmenes
`GET /` y `GET /calcular`	formulario	interfaz e informe de heterogeneidad			

`/consulta` no devuelve un CATE individual. `/heterogeneidad` tampoco: usa la proyección lineal almacenada y advierte el alcance condicional del IC. `/efecto_individual` conserva el núcleo de respuesta y añade la deprecación. El formulario de `GET /` o `GET /calcular` es la vía prevista para construir un perfil con las 25 `X`, en lugar de escribir ese JSON a mano.

Ejemplo ilustrativo de `POST /consulta` con una sola covariable de corte:

```
curl --fail-with-body -X POST http://127.0.0.1:8000/consulta \\
  -H 'Content-Type: application/json' -d '{"bw":1.0}'
```

`bw = 1.0` no es un perfil recomendado: comprueba primero que el valor está dentro de `rangos_covariables.bw` del bundle cargado. Una respuesta normal contiene, entre otras claves verificadas, esta forma abreviada:

```
{
  "estado": "respuesta",
  "efecto": …,              \# media redondeada del estrato, no efecto individual
  "ic95": \[…, …\],
  "estado\_puerta": …,
  "version": …
}
```

No se promete aquí una cifra: depende del estrato y de la versión cargada. Si `bw` falta o queda fuera de rango, la abstención incluye `estado`, `motivo`, `estado_puerta` y `version`, y **no contiene `efecto` ni `ic95`**.

Soporte y validación HTTP

La abstención por soporte compara cada covariable suministrada con su **rango marginal** de entrenamiento. Estar dentro de todos los rangos no garantiza soporte multivariable ni comparabilidad entre tratados y controles para el vector completo.

Los endpoints reciben diccionarios JSON, pero no hay un esquema Pydantic por covariable. Un string como `{«bw»:«alto»}` puede terminar en **HTTP 500** al convertirlo a número, en vez de producir una abstención. El cliente debe validar tipos finitos y distinguir errores HTTP de abstenciones causales.

Decisión de diseño

El servicio depende de JSON sencillo, no de los datos ni de las bibliotecas de estimación. Esto reduce la superficie de despliegue y hace explícita la versión que origina cada respuesta. La abstención evita fabricar precisión fuera de los contratos.

Límites

No hay autenticación, TLS ni validación exhaustiva de esquema. La integridad no equivale a seguridad frente a un adversario. La caché exige disciplina de reinicio y el fallback al manifest puede reflejar una puerta anterior al monitor.

Comprobación y práctica

Servicio, sin promoción: arranca con `arrancar.sh` o `arrancar.ps1` y consulta:

```
curl --fail-with-body http://127.0.0.1:8000/salud
curl --fail-with-body http://127.0.0.1:8000/efecto
```

Estas peticiones leen el servicio; no crean versiones. `/docs` permite inspeccionar los contratos.

En la copia, usa perfiles construidos desde los rangos del propio bundle. Prueba por separado: falta de `bw`, una covariable fuera de rango, un perfil completo y un tipo no numérico. Explica cuál produce abstención y cuál puede producir error HTTP. No fuerces casos de integridad sobre los artefactos de referencia; `humo.py` ya usa copias temporales para sus casos defectuosos.

13. Pruebas, evidencia, instalación y Docker

Pruebas sin promoción

Desde la raíz, en macOS/Linux:

```
SIC\_ESCENARIO=synthetic/ihdp .venv/bin/python datos.py
SIC\_ESCENARIO=synthetic/ihdp .venv/bin/python pruebas\_estimacion.py
SIC\_ESCENARIO=synthetic/ihdp .venv/bin/python humo.py
shasum -a 256 -c evidencia/SHA256SUMS
```

En Linux, si se usa la utilidad habitual de GNU en lugar de `shasum`:

```
sha256sum -c evidencia/SHA256SUMS
```

En Windows PowerShell:

```
\$env:SIC\_ESCENARIO = "synthetic/ihdp"
.\\.venv\\Scripts\\python.exe datos.py
.\\.venv\\Scripts\\python.exe pruebas\_estimacion.py
.\\.venv\\Scripts\\python.exe humo.py
```

`Get-FileHash -Algorithm SHA256 <archivo>` permite comparar un archivo individual con `evidencia/SHA256SUMS`, pero no procesa automáticamente toda la lista.

Estas comprobaciones **no promueven una versión**. `pruebas_estimacion.py` está escrita para IHDP, no para jobs. Comprueba fórmulas, identificación declarada, monitor, cobertura en memoria y contratos del artefacto. Las comprobaciones vinculadas a `artifacts/estimacion.json` son parciales: se ejecutan solo si ese archivo existe; su existencia activa además exigencias de esquema y coherencia. `humo.py` necesita un champion íntegro y usa `TestClient`, sin abrir un puerto. Sus mutaciones de error viven en memoria o directorios temporales.

Con el entorno activado, los atajos `make verificar`, `make pruebas` y `make humo` son equivalentes y tampoco promueven. Recuerda que el `Makefile` resuelve `python3` mediante `PATH`.

Evidencia conservada

`evidencia/README.md` relaciona cifras, comandos y archivos. `shasum` o `sha256sum` comprueba diferencias respecto de `SHA256SUMS` no autentica el origen.

El orden de una revisión útil es:

1. `01_datos.txt`: esquema, hashes y pared;
2. `02_estimacion.txt`: estimadores, AIPW y diagnósticos;
3. `03_sensibilidad.txt` y `04_placebo.txt`: alcance de sensibilidad y placebos;
4. `06_registro.txt`: versión y alias;
5. `05_puerta.txt`: producción y estrés;
6. `07_servicio.txt` y `09_pruebas.txt`: contratos;
7. `10_cobertura.*`: otro DGP, con alcance separado.

La cobertura Monte Carlo no evalúa IHDP ni jobs. Sirve para examinar el comportamiento de los estimadores en el DGP propio de `cobertura.py`, no para regenerar el dataset ni probar identificación.

Lectura conjunta de cobertura

Cobertura, sesgo, RMSE y longitud deben leerse juntos: una cobertura cercana al nominal no implica estimador eficiente ni IC corto, y un IC corto puede tener sesgo o mala cobertura. En la corrida conservada propia (`n = 500`, `R = 100`), AIPW cubre **0,97**, OLS ajustado **0,96** y el ingenuo **0,00**. Pero OLS tiene RMSE **0,098**, menor que **0,114** de AIPW, y longitud media **0,389**, menor que **0,442**: AIPW no resulta superior en todas las dimensiones.

El DGP favorable a los modelos usados —resultado lineal y asignación logística, con confusores observados y solape moderado — no prueba comportamiento ante no linealidad, confusión oculta o positividad extrema. Con `R = 100` la precisión es limitada; desviarse ±1 error Monte Carlo es una descripción, no un test formal. El error Monte Carlo 0 de cobertura 0 es degenerado y no aporta certeza. Una sola corrida sobre IHDP tampoco estima cobertura real. Las cifras proceden de `artifacts/cobertura.json`/`evidencia/10_cobertura.json`, no de una validación ejecutada para esta guía.

Comandos que escriben o promueven

En la **copia de práctica**, la secuencia orquestada con escenario explícito es:

```
SIC\_ESCENARIO=synthetic/ihdp make demo
```

Para varios atajos consecutivos en macOS/Linux también sirve `export SIC_ESCENARIO=synthetic/ihdp` en PowerShell, `$env:SIC_ESCENARIO = «synthetic/ihdp»`, seguido de `make` solo si está disponible, o de los comandos directos con `.\.venv\Scripts\python.exe`. No uses `make -j`: la cadena exige el orden de sus prerrequisitos y no debe paralelizarse.

`make demo` equivale a `verificar → estimar → registrar → ventanas → humo`. Verifica los CSV existentes, escribe estimaciones, crea una nueva versión, **promueve un champion**, escribe el monitor y ejecuta humo. No invoca un generador IHDP.

```
make evidencia
```

Además de ejecutar etapas que escriben artefactos y promueven, **sobrescribe salidas de `evidencia/`** y recalcula `SHA256SUMS`. El objetivo `evidencia` no ejecuta `make dag` antes de hashear los SVG: si se quieren actualizar esos SVG, `make dag` es una operación de escritura independiente que debe decidirse explícitamente.

Otros efectos que conviene no confundir con lectura:

Comando	Efecto	`make estimar`	reemplaza los tres artefactos de estimación; no promueve
`make registrar`	crea y promueve una versión	`make ventanas`	escribe `monitor_estado.json` no promueve
`make cobertura`	escribe cobertura de otro DGP; no genera IHDP ni promueve	`make dag`	regenera SVG con Graphviz
`make cierre`	compila el manual Typst existente	`make limpiar`	**borra todo `artifacts/`**, incluidos versiones y champion

No uses `make limpiar && make demo` como supuesta comprobación: combina borrado completo con regeneración y promoción. Tampoco uses `make demo` o `make evidencia` para «ver si todo está bien» en la entrega de referencia.

Instalación: alcance y verificación

La instalación descrita en la sección 3 termina con `uv pip check` y la versión del intérprete. No demuestra por sí sola que datos, bundle o API sean correctos. La secuencia mínima de auditoría separa:

1. coherencia del entorno;
2. `datos.py`
3. `pruebas_estimacion.py`
4. `humo.py`
5. hashes de evidencia;
6. si se arranca el servicio, `/salud` y `/efecto`.

Ninguno de esos pasos exige promover. Las cifras de `v0008` describen evidencia existente y no sustituyen la comprobación en otro entorno.

Docker

El `Dockerfile` instala únicamente lo necesario para servir JSON. La imagen incluye código de servicio, registro, sensibilidad y healthcheck, más `artifacts/` **no incluye CSV, DoWhy ni pandas**. No deserializa `pickle` o `joblib` y ejecuta como usuario no root.

La variable de imagen y Compose selecciona `synthetic/ihdp`. `healthcheck.py` exige conjuntamente:

- `estado == «ok»`
- `integridad_valida == true`
- `estado_puerta == «pasa»`
- `version == SIC_VERSION_ESPERADA`.

Por tanto, una API que devuelve números bajo `alerta`, `reentrenar` o `revisar_identificacion` puede seguir respondiendo, pero **no supera el healthcheck de Docker**, que exige `pasa`.

`docker-compose.yml` fija actualmente `SIC_VERSION_ESPERADA=v0008`. Si cambia el champion incluido en la imagen, hay que actualizar ese valor y reconstruir la imagen; modificar el host no cambia un contenedor ya construido.

Construir o arrancar Docker es una operación de **servicio**, no de promoción dentro del repositorio fuente. La imagen captura los artefactos presentes al construirla, de modo que hay que auditar primero qué champion se está empaquetando; no crea una versión nueva ni promueve.

Ruta mínima desde la raíz:

```
docker compose up --build
curl --fail-with-body http://127.0.0.1:8000/salud
docker compose ps
docker compose down
```

`up` deja normalmente la terminal ocupada: ejecuta las consultas y `ps` desde otra. Antes de `/efecto`, comprueba `estado = ok`, `integridad_valida = true`, `version` esperada y `estado_puerta`. En Windows PowerShell usa `curl.exe` (para no invocar el alias de `curl`) o `Invoke-RestMethod http://127.0.0.1:8000/salud`.

Cautela de verificación: el README documenta revisión estática de Windows/PowerShell y de Compose, no ejecución nativa verificada en Windows ni ejecución verificada de Compose. Las instrucciones anteriores no deben presentarse como una prueba realizada al actualizar esta guía.

Práctica autónoma

Formula una tabla con columnas «comando», «lee», «escribe», «promueve», «requiere champion» y «reversible». Rellénala desde el `Makefile`, no desde los nombres de los objetivos. Después ejecuta únicamente las comprobaciones sin promoción en la copia y contrasta tu tabla con los resultados.

14. Adaptación a otro escenario

Adaptar no consiste en cambiar un nombre de carpeta. Hay que rehacer el contrato causal y todos los puntos de configuración antes de estimar.

1. Define pregunta y diseño

Declara unidad, población, tratamiento, resultado, ATE u otro estimando, momento de medición de `X` y supuestos de consistencia/SUTVA. Justifica el conjunto de ajuste con el diseño. No heredes el DAG de IHDP por conveniencia.

2. Prepara entradas observadas

Crea el escenario bajo `data_benchmark_causal/causal_benchmark_synthetic/datasets/` con:

- `observed_train.csv`, `observed_validation.csv` y `observed_test.csv`
- `splits.json` con hashes y metadatos;
- `identification.json` coherente con la estrategia declarada;
- documentación de procedencia y soporte.

La verdad reservada no debe entrar en entrenamiento, bundle o servicio. Si no dispones de generador, documenta ese límite igual que se hace con IHDP.

3. Actualiza configuración real

Revisa, como mínimo:

- `COVARIABLES` y `RESULTADO` en `datos.py`
- `CORTES`, `UNIDAD`, `ETIQUETA_UNIDAD`, `RUIDO_TRAMPA` y `VARIABLE_PLACEBO` en `estimacion.py`
- `UMBRALES` en `registro.py`
- `CONFIG` en `monitor.py`
- supuestos, población, uso previsto y limitaciones de la tarjeta;
- perfiles y pruebas específicas, contratos de servicio y documentación.

Un corte de heterogeneidad debe justificarse antes de mirar efectos. Un placebo debe ser defendible para el nuevo diseño; `momage` no tiene sentido fuera de IHDP. Los umbrales son decisiones de operación iniciales, no constantes universales.

4. Revisa identificación y estimación

Comprueba que el DAG representa el diseño y que `valid_adjustment_sets` no se usa como sustituto de una justificación causal. Evalúa si OLS por brazo y logística son razonables; cross-fitting no corrige automáticamente una mala forma funcional. Si cambias el estimando, revisa también E-value, valor de robustez, Manski, estratos y endpoints: no todos son intercambiables.

5. Trabaja en la copia y registra de forma explícita

Primero usa fixtures y pruebas. Después, si decides escribir artefactos, separa estimación, registro, monitor y humo para inspeccionar cada salida. Una promoción crea una versión nueva; no sobrescribas la evidencia anterior para dar apariencia de continuidad.

Actualiza `SIC_VERSION_ESPERADA` cuando empaquetes otro champion. Reinicia el servicio al cambiar escenario o versión.

6. Reescribe la evidencia de alcance

Documenta qué se ejecutó realmente, qué quedó sin probar y qué cifras pertenecen a qué versión. No transfieras el ATE, el E-value, las cotas, los resultados de placebos ni la cobertura de IHDP al nuevo escenario.

15. Errores frecuentes

Error	Causa	Corrección	Ejecutar sin `SIC_ESCENARIO` y estudiar otro dataset	el default de `rutas.py` es `synthetic/jobs`	fijar `synthetic/ihdp` antes de cada comando o usar los scripts de la entrega
Usar `python3` global con `.venv` instalado	el `Makefile` y `uvicorn` dependen de `PATH`	activar el entorno o usar rutas directas	Suponer que `setup` valida o promueve	su alcance termina en entorno y dependencias	ejecutar comprobaciones separadas; no hace falta promover
Ejecutar `make demo` como lectura	registra y promueve además de estimar	usar `datos.py`, pruebas, humo y hashes para comprobar	Ejecutar `make evidencia` para conservar evidencia	sobrescribe salidas y hashes, y promueve	leer la evidencia existente; regenerar solo en copia
Usar `make limpiar` como limpieza de cachés	elimina todo `artifacts/`	no usarlo en la entrega; nunca encadenarlo con `demo` como rutina	Intentar regenerar IHDP con `cobertura.py`	ese archivo simula otro DGP	partir de los CSV incluidos y conservar el límite del generador ausente
Leer recuentos impresos como un nuevo conteo	`datos.py` imprime `n_rows` desde `splits.json`	distinguir metadatos declarados de hashes recomputados	Tratar la pared como seguridad criptográfica	solo exige ausencia literal de `truth_train.csv`	revisar también accesos, nombres alternativos y flujo de verdad
Interpretar DoWhy como prueba de ignorabilidad	solo opera sobre el grafo y metadatos declarados	defender el diseño y reportar sensibilidad sin afirmar identificación empírica	Llamar holdout predictivo a la validación por particiones	cada partición reestima AIPW	describirla como chequeo de estabilidad del ATE
Diagnosticar tras acotar y olvidar el origen OOF	rango, solape y recuentos se conservan antes de acotar	reportar la fuente y el momento de cada diagnóstico	Confundir `recortado_n` con filas excluidas	`np.clip` modifica propensiones y conserva las 448 filas	llamarlo acotación y contrastar ESS/SMD con solape y extremos
Confundir `score_post` con placebo	está construido con `Y` y demuestra fuga	usar las permutaciones de `D` o del pseudo-resultado para calibración	Interpretar estratos o proyección como CATE individual	ambos resumen scores mediante otro estimando	conservar etiquetas, IC condicional y advertencias
Llamar RR literal al E-value continuo	usa aproximación por efecto estandarizado	declarar la aproximación y sus límites	Atribuir el RV exacto a AIPW	Cinelli–Hazlett se calcula desde OLS	etiquetarlo como `valor_robustez_ols`
Presentar Manski como ATE puntual	sin ignorabilidad hay identificación parcial	reportar cotas y fuente de soporte	Afirmar que el manifest cubre el monitor	solo hashea estimación y tarjeta	comprobar además `monitor.version`
Prometer actualización atómica de todo el alias	solo `registro/champion` usa rename atómico	reconocer escritura previa no transaccional del alias	Esperar rollback si monitor falla	registro y monitor son etapas independientes	revisar la versión ya promovida y el estado resultante
Asumir que toda alerta bloquea números	el núcleo bloquea integridad rota o `falla` exacta	distinguir puerta de API y criterio más estricto del healthcheck	Dejar que `peor_accion` determine producción	producción depende solo de `SIC_VENTANA`	leer `accion` y `estado_puerta` de la ventana seleccionada
Suponer que ±20 % reajusta el modelo	solo reclasifica medidas guardadas	describirlo como sensibilidad local de umbrales	Confiar en rangos marginales como soporte conjunto	no describen la distribución multivariable	reconocer extrapolación posible incluso dentro de cada rango
Esperar abstención para cualquier JSON inválido	no existe esquema Pydantic por covariable	validar tipos en el cliente; un string puede causar HTTP 500	Olvidar la caché tras cambiar champion	una carga válida permanece en el proceso	reiniciar y verificar la versión en `/salud`
Cambiar champion en Docker sin actualizar la versión esperada	Compose conserva `SIC_VERSION_ESPERADA`	actualizar, reconstruir y revisar healthcheck			

16. Checklist final

Comprensión causal

- ☐ Puedo enunciar ATE, población, unidad y tratamiento sin convertirlos en recomendación.
- ☐ Sé por qué el ajuste backdoor depende del diseño y no de una prueba automática.
- ☐ Distingo identificación algebraica, estabilidad entre particiones e ignorabilidad.
- ☐ Explico qué no demuestran DoWhy, placebos, E-value, valor de robustez y Manski.
- ☐ No presento estratos ni proyección como CATE individual ni el R^2 como porcentaje de heterogeneidad causal.
- ☐ Para afirmar heterogeneidad exijo un contraste entre subgrupos con incertidumbre, no solo dos hojas distintas.
- ☐ Distingo el ATE superpoblacional de un SATE de las 448 filas y adapto la inferencia si cambia el objetivo.

Datos y estimación

- ☐ `SIC_ESCENARIO=synthetic/ihdp` está explícito.
- ☐ Los roles contienen `treatment`, `outcome` y las 25 covariables correctas.
- ☐ Los splits verifican hashes y la ausencia literal de `truth_train.csv` se mantiene.
- ☐ Sé que el generador IHDP no está incluido y no intento sustituirlo con cobertura.
- ☐ Puedo derivar el score AIPW, su SE y su IC.
- ☐ Sé que `np.clip` acota propensiones sin eliminar filas y que `recortado_n` no cuenta exclusiones.
- ☐ Los rangos, solape y recuentos se sitúan antes de acotar; ESS y SMD ponderado usan la propensión acotada.
- ☐ Leo ESS, solape, extremos y SMD juntos, sin confundir mediciones *in-sample* del monitor con OOF del champion.
- ☐ Distingo IC normal asintótico de cobertura exacta y particiones reentrenadas de un *holdout* con champion congelado.
- ☐ Las cifras históricas se atribuyen a su versión, sin tratar `v0008` como garantía.
- ☐ Evalúo conjuntamente estimando, magnitud/relevancia, precisión, identificación, soporte, estabilidad, sensibilidad y calibración, sin convertirlos en un *score*.
- ☐ No deduzco relevancia práctica solo del IC ni causalidad de signo, `p`, placebos o solapes de IC.
- ☐ Limito mis conclusiones a compatibilidad condicional en el escenario sintético; no las presento como recomendación ni como evidencia sobre niños reales.

Registro, monitor y servicio

- ☐ La tarjeta contiene los 15 campos del esquema 1.2.
- ☐ Sé qué archivos hashea el manifest y que el monitor queda fuera de él.
- ☐ Comprendo alias, rename atómico limitado y ausencia de rollback.
- ☐ Puedo ordenar las cinco acciones y relacionar cada señal con su acción.
- ☐ Solo la ventana `SIC_VENTANA` fija producción; `peor_accion` es resumen.
- ☐ La sensibilidad ±20 % reclasifica sin reajustar AIPW.
- ☐ El servicio usa solo el bundle JSON y su caché se controla mediante reinicio.
- ☐ Distinguo `falla`, alertas operativas, abstención por soporte y errores HTTP.
- ☐ El healthcheck exige estado, integridad, `pasa` y versión esperada.

Operación y evidencia

- ☐ He clasificado cada comando como lectura, comprobación, escritura, promoción o servicio.
- ☐ Las comprobaciones sin promoción usan el intérprete correcto.
- ☐ `make demo`, `make evidencia` y `make limpiar` no se usan como lectura.
- ☐ Los experimentos y mutaciones se hacen en copia temporal, no en la entrega de referencia.
- ☐ Los hashes de evidencia verifican y sus límites de autenticación están claros.
- ☐ Docker empaqueta el champion revisado y `SIC_VERSION_ESPERADA` coincide.
- ☐ Al adaptar otro escenario se rehacen pregunta, esquema, DAG, cortes, placebos, umbrales, pruebas y documentación.

17. Referencias del repositorio

- README principal: puesta en marcha, alcance y mapa general.
- Pregunta causal de IHDP: estimando, diseño, supuestos, resultados históricos y límites.
- Manual en Typst: fuente editable del manual del sistema.
- Manual en PDF: versión de lectura del mismo manual.
- Guía de evidencia: trazabilidad entre comandos, salidas, cifras y hashes.
- Código de la cadena: rutas.py, datos.py, estimacion.py, sensibilidad.py, registro.py, monitor.py, servicio_core.py y servicio.py.
- Comprobaciones: pruebas_estimacion.py, humo.py y healthcheck.py.
- Orquestación y empaquetado: Makefile, Dockerfile y docker-compose.yml.

Referencias metodológicas citadas por la implementación y la documentación del proyecto: Hill (2011) para el benchmark IHDP; VanderWeele y Ding (2017), junto con la aproximación de Chinn (2000), para el E-value continuo; Cinelli y Hazlett (2020) para el valor de robustez bajo OLS; Manski para identificación parcial con resultado acotado. Estas referencias explican los métodos, pero no convierten los diagnósticos del repositorio en pruebas de ignorabilidad.